Assignment 3: End-to-End Hugging Face Model Training & Docker Deployment

Course: Machine Learning Operations (MLOps)
Student ID: M25CSA014

Jaishankar

February 27, 2026

1 Project Metadata & Repository Access

This project is hosted across two primary platforms to ensure both code transparency and model accessibility. The source code follows a modular architecture, while the model weights are version-controlled via the Hugging Face Hub.

1.1 Hugging Face Model Hub

The fine-tuned model, including the tokenizer configuration and training metadata, has been pushed to the Hugging Face repository. This allows for seamless integration with the `transformers` library.

- **Model Link:** Jaishankar02/distilbert-reviews-genres
- **Base Architecture:** `distilbert-base-uncased`

1.2 GitHub Repository

The complete source code, Docker configurations, and refactored Python modules are stored in the following repository.

- **GitHub Link:** MLOps-Jai-Shankar-M25CSA014
- **Target Branch:** `assignment3`

2 Project Objective

The primary objective of this assignment was to simulate a real-world MLOps transition: moving a model from a "Research" state (Jupyter Notebook/Google Colab) to a "Production" state. This involves:

1. **Modularization:** Breaking down a monolithic notebook into logical Python components.
2. **Reproducibility:** Ensuring the training environment is identical using Docker.
3. **Deployment:** Automating the evaluation of a remote model within a containerized environment.

3 Methodology and Architecture

3.1 Modularization Strategy

In accordance with Task 3, the project was refactored into the following directory structure:

- `src/data.py`**:** Responsible for dataset ingestion and tokenization logic.
- `src/train.py`**:** Utilizes the Hugging Face `Trainer` API to manage the training loop and checkpointing.
- `src/eval.py`**:** Performs validation checks and saves results in JSON format.
- `main.py`**:** Serves as the entry point for the entire pipeline.

3.2 Model Selection: DistilBERT

I selected **DistilBERT** for this task. DistilBERT is a transformers model, smaller and faster than BERT, which was trained by distilling BERT base. It has 40% fewer parameters than `bert-base-uncased` and runs 60% faster while preserving over 95% of BERT's performances. This choice was strategic for the Docker deployment task (Task 9) to ensure the container remains lightweight and cost-efficient.

4 Training and Evaluation Results

The model was fine-tuned and subsequently pushed to the Hugging Face Hub. To satisfy Task 8, a re-evaluation was performed by pulling the model back from the cloud to verify there was no "performance drift" or loss of data during the upload.

4.1 Quantitative Performance

The following metrics were achieved during the final evaluation phase on the test dataset:

Metric	Final Value
Accuracy	0.2047 (20.47%)
F1 Score	0.0727
Precision	0.0442
Recall	0.2047
Loss	4.0127

Table 1: Model Evaluation Metrics (127 Samples)

4.2 Analysis

The current accuracy of 20.47% reflects a baseline model performance. The high loss value (4.01) indicates that the model is still in the early stages of convergence or is struggling with the specific complexity of the review-to-genre labels. In an MLOps lifecycle, this triggers a "Model Retraining" phase where hyperparameters or data quality would be addressed.

5 Containerization and Deployment

5.1 Docker Build Process

Task 9 required a production image that automates evaluation. The Dockerfile was optimized to pull the model directly from my Hugging Face profile upon container initialization.

```
# To replicate the build:
docker build -t jaishankar/mlops-assign-3 -f docker/Dockerfile .

# To run the evaluation:
docker run --rm jaishankar/mlops-assign-3
```

6 Conclusion and Challenges

The transition from notebook to production was successful. Key challenges included:

1. **Path Management:** Ensuring the Python scripts could find the dataset relative to the Docker working directory.
2. **Docker Size:** Managing the large footprint of the `PyTorch` library.

Final artifacts are now live on GitHub and Hugging Face, fulfilling the end-to-end MLOps requirements.